

# 粒子群算法

## (Particle Swarm Optimization, PSO)

理解粒子群算法的思想来源，熟练掌握粒子群算法的设计流程和实例计算。掌握粒子群算法中各参数的含义和设置范围，了解粒子群算法的各种改进角度和改进方法，并了解粒子群算法的实际应用。

# PSO

自然界中的鸟群、兽群、鱼群等在迁徙、捕食过程中，往往表现出高度的组织性和规律性。把动物的**群体智能行为**与人类的**社会认知特性**相结合成为 PSO 算法的思想来源。

- 1987 年，Reynolds 实现了**鸟群运动的计算机可视化仿真**。
- 1990 年，动物学家 Heppner 和 Grenander 也对**动物的群体活动规律**进行了研究，包括大规模群体同步聚合，突然地改变方向，规律的分散与重组等相关的机制和潜在的规律。
- 20 世纪 70 年代，Wilson 指出："至少在理论上，在群体觅食的过程中，群体中的**每一个个体都会受益于所有成员在这个过程**中所发现和累积的经验"。
- Eberhart 指出，除了考虑模拟生物的群体活动之外，更重要的是**融入了个体认知和社会影响**等社会心理学的理论。<sup>2</sup>







# PSO

- 粒子群优化算法是进化计算的一个分支，是一种模拟自然界的生物活动以及群体智能的**随机搜索算法**。
- 在借鉴自然界生物群体活动的认识以及这些活动行为计算机可视化仿真的基础上，将社会心理学上的**个体认知、社会影响、群体智慧**等思想融入到组织性和规律性很强的群体行为中，开发一个可以用于工程实践的优化工具。
- 粒子群优化算法结合了**动物的群体行为特性**以及**人类社会的认知特性**。

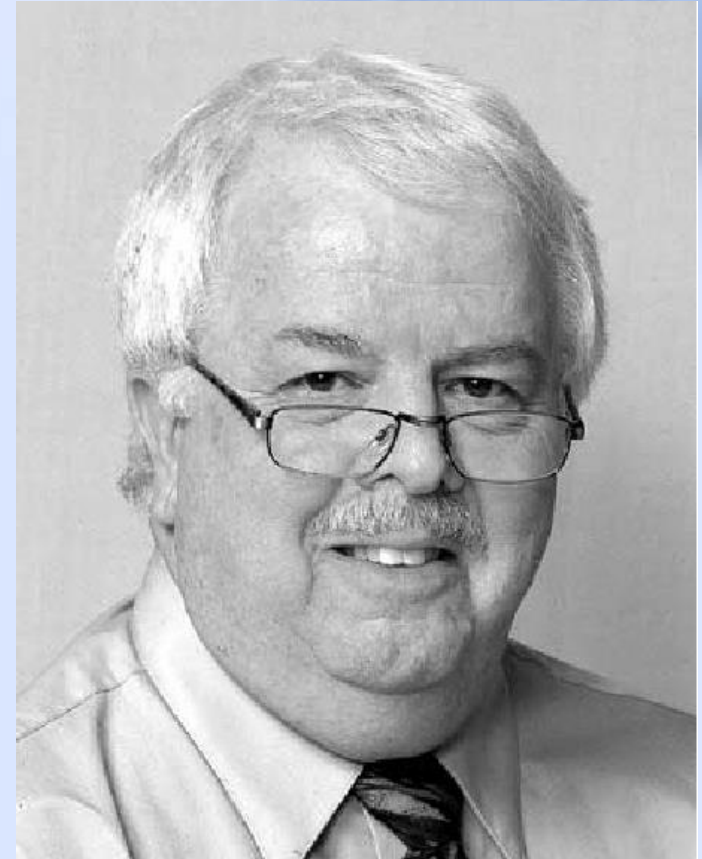
# PSO

- 粒子群算法(particle swarm optimization, PSO)由 Kennedy和 Eberhart 在1995年提出, 该算法模拟**鸟群集体飞行觅食**的行为, 鸟之间通过**集体协作**使群体达到最优目的, 是一种基于Swarm Intelligence的优化方法。
- 同遗传算法类似, 也是一种**基于群体迭代**的, 但并没有遗传算法用的交叉以及变异, 而是粒子在**解空间追随最优的粒子**进行搜索。
- PSO的优势在于简单容易实现同时又有深刻的智能背景, 既适合科学研究, 又特别适合工程应用, 并且没有许多参数需要调整。



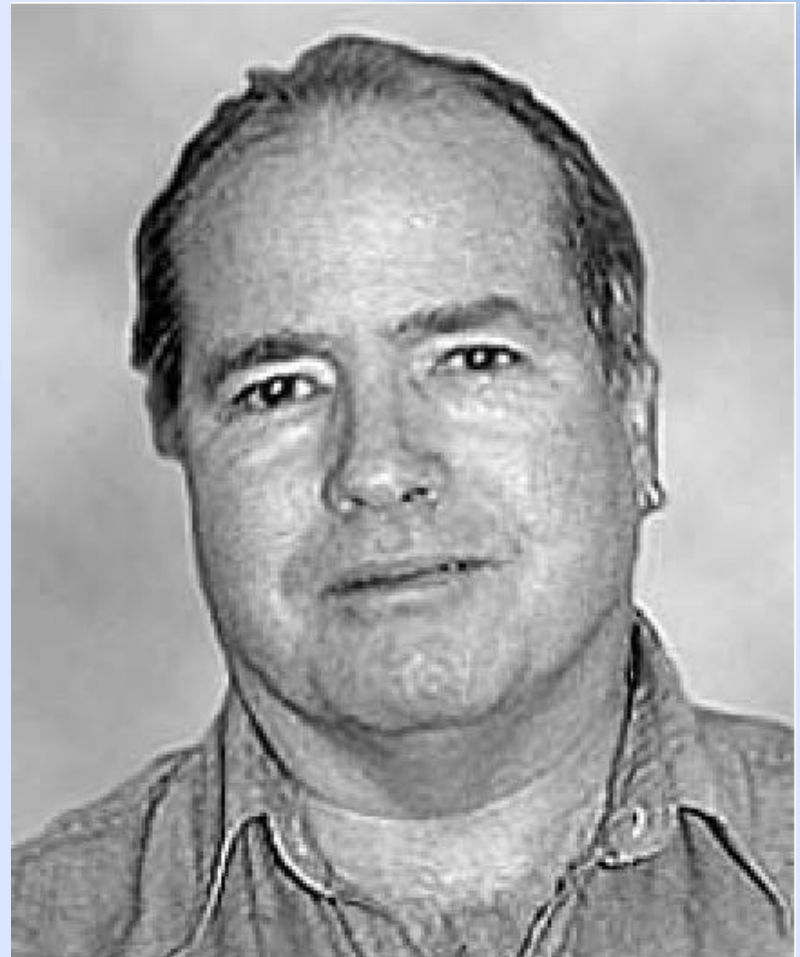
# PSO

- **Russell C. Eberhart** received the Ph.D. degree in **electrical engineering** from Kansas State University, Manhattan. He is the Chair and Professor of Electrical and Computer Engineering, Purdue School of Engineering and Technology, Indiana University–Purdue University Indianapolis (IUPUI), Indianapolis, IN. He is coeditor of **Neural Network PC Tools**(1990), coauthor of **Computational Intelligence PC Tools** (1996), coauthor of **Swarm Intelligence**(2001), **Computational Intelligence: Concepts to Implementations**(2004).
- He has published over 120 technical papers. Dr. Eberhart was awarded the IEEE Third Millennium Medal. In 2002, he became a Fellow of the American Institute for Medical and Biological Engineering.



# PSO

- **James Kennedy** received the Ph.D. degree from the University of North Carolina, Chapel Hill, in 1992. He is with the U.S. Department of Labor, Washington, DC. He is a **Social Psychologist** who has been working with the particle swarm algorithm since 1994. He has published dozens of articles and chapters on particle swarms and related topics, in computer science and social science journals and proceedings. He is a coauthor of **Swarm Intelligence** (San Mateo, CA: Morgan Kaufmann, 2001), with R.C. Eberhart and Y. Shi, now in its third printing.





# PSO

- 1987年Reynolds实现了鸟群社会系统的仿真研究(boids系统)，一群鸟在空中飞行，每个鸟遵守以下三条规则：
  - 飞离最近的个体，避免与相邻的鸟发生碰撞冲突；
  - 飞向目标；
  - 尽量向自己所认为的群体中心靠近。
- 通过使用这三条规则，boids系统出现了非常逼真的群体聚集行为，鸟成群地在空中飞行，当遇到障碍时它们会分开绕行而过，随后又会重新形成群体。

# 小鸟通过什么样的机制找到食物呢？

通过各自探索与群体合作最终发现食物的所在位置。

- 随机地飞行觅食，它们不知道食物所在的具体位置，但是有一个间接的机制会让小鸟知道它当前位置离食物的距离(例如食物香味的浓淡等)。
- 在飞行的过程中不断地记录和更新它曾经到达的离食物最近位置。
- 通过信息交流的方式比较大家所找到的最好位置，得到一个当前整个群体已经找到的最佳位置。
- 结合自身经验和整个群体的经验，调整自己的飞行速度和所在位置，不断地寻找更加接近食物的位置，最终使得群体聚集到食物位置。

# PSO

- 鸟群中的每个小鸟被称为一个“**粒子(Particle)**”。
- 随机产生一定规模的粒子，作为优化问题搜索空间中的**有效解**，每个解都是搜索空间中的一只鸟。
- 每个粒子都具有**速度和位置**，用于决定他们飞翔的方向和距离。
- 所有的粒子都有一个由问题定义的**适应度函数**确定粒子的适应值。
- 粒子通过追随最优粒子的位置在解空间中搜索，**不断更新自己的飞行速度和下一个位置**。让粒子在搜索空间中探索 and 开发，最终找到全局最优解。



# PSO

表 6.1 鸟群觅食和粒子群优化算法的基本定义对照表

| 鸟 群 觅 食   | 粒子群优化算法                                              |
|-----------|------------------------------------------------------|
| 鸟群        | 搜索空间的一组有效解(表现为种群规模 $N$ )                             |
| 觅食空间      | 问题的搜索空间(表现为维数 $D$ )                                  |
| 飞行速度      | 解的速度向量 $\mathbf{v}_i = [v_i^1, v_i^2, \dots, v_i^D]$ |
| 所在位置      | 解的位置向量 $\mathbf{x}_i = [x_i^1, x_i^2, \dots, x_i^D]$ |
| 个体认知与群体协作 | 每个粒子 $i$ 根据自身历史最优位置和群体的全局最优位置更新速度和位置                 |
| 找到食物      | 算法结束,输出全局最优解                                         |

# PSO算法的基本流程

- 粒子群优化算法(PSO) 要求每个粒子在飞行过程中维护两个向量，就是速度向量  $v_i = [v_i^1, v_i^2, \dots, v_i^D]$  和位置向量  $x_i = [x_i^1, x_i^2, \dots, x_i^D]$
- 每次迭代中，粒子通过跟踪两个“最优值”来更新自己。一个就是粒子本身所找到的最优解，即历史最优位置 *pBest*。另一个是整个种群目前找到的全局最优解 *gBest*，就是所有粒子的 *pBest* 中最优的那个。(另外,也可以不用整个种群而只是用其中一部分的邻居)
- 迭代过程中，通过速度更新公式和位置更新公式不断地进化而得到全局最优解，因此，PSO 的原理和机制更加简单，算法实现也相对容易，运行效率更高。

# PSO算法的主要步骤

- (1) **初始化**。初始化所有的个体(粒子)的**速度和位置**，并且将个体的历史最优  $pBest$  设为当前位置，而群体中最优的个体作为当前的  $gBest$ 。
- (2) **计算适应值**。在每一次迭代中，计算各个粒子的适应度函数值（适应度函数的表达式可根据具体问题给出）。
- (3) **更新历史最优值**。对于每个粒子，如果该粒子当前的适应度函数值比其历史最优值要好，那么历史最优将会被当前位置所替代。
- (4) **更新全局最优值**。如果某粒子的历史最优比全局最优要好，那么全局最优将会被该粒子的历史最优所替代。



(5) **更新速度和位置**。对每个粒子  $i$  的第  $d$  维的速度和位置分别按照下面的公式进行更新。

$$\begin{aligned} v_i^d &= \omega \times v_i^d \\ &+ c_1 \times rand_1^d \times (pBest_i^d - x_i^d) \\ &+ c_2 \times rand_2^d \times (gBest^d - x_i^d) \end{aligned}$$

$$x_i^d = x_i^d + v_i^d$$

- 设搜索空间共有  $D$  维,粒子个数为  $n$ 。
- 第  $i$  个粒子位置表示为向量  $x_i=(x_i^1, x_i^2, \dots, x_i^D)$ ;  
第  $i$  个粒子速度 (位置变化率) 表示为向量  $v_i=(v_i^1, v_i^2, \dots, v_i^D)$
- 第  $i$  个粒子在 “飞行” 历史中的最优位置(即历史最优解)为  $pBest_i=(pBest_i^1, pBest_i^2, \dots, pBest_i^D)$ ,
- 其中第  $g$  个粒子的历史最优位置  $pBest_g$  为所有  $pBest_i$  ( $i=1, \dots, n$ ) 中的最优值, 记为  $gBest$

$$\begin{aligned} v_i^d &= \omega \times v_i^d \\ &+ c_1 \times rand_1^d \times (pBest_i^d - x_i^d) \\ &+ c_2 \times rand_2^d \times (gBest^d - x_i^d) \end{aligned}$$

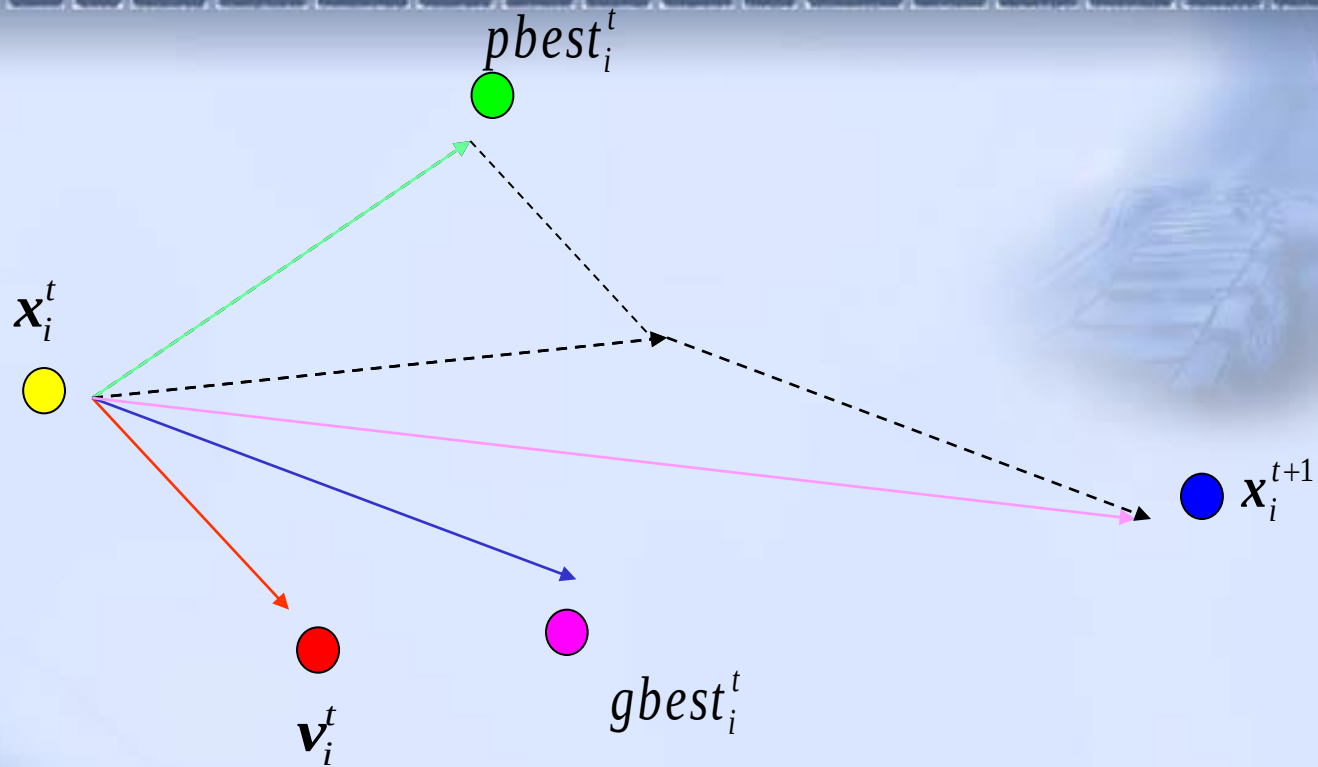
$$x_i^d = x_i^d + v_i^d$$

$\omega$  是惯量权重 (Inertia Weight)，一般初始化为 0.9，然后随着迭代过程线性递减到 0.4；

$c_1, c_2$  是加速系数 (Acceleration Coefficients)，也称学习因子)，传统上都是取固定值 2.0；

$rand()$  为 [0, 1] 之间的随机数；

位置更新必须是合法的，所以在每次进行更新之后都要检查更新后的位置是否在问题空间之中



$$\begin{aligned}
 v_i^d &= \omega \times v_i^d \\
 &+ c_1 \times rand_1^d \times (pBest_i^d - x_i^d) \\
 &+ c_2 \times rand_2^d \times (gBest^d - x_i^d)
 \end{aligned}$$

$$x_i^d = x_i^d + v_i^d$$

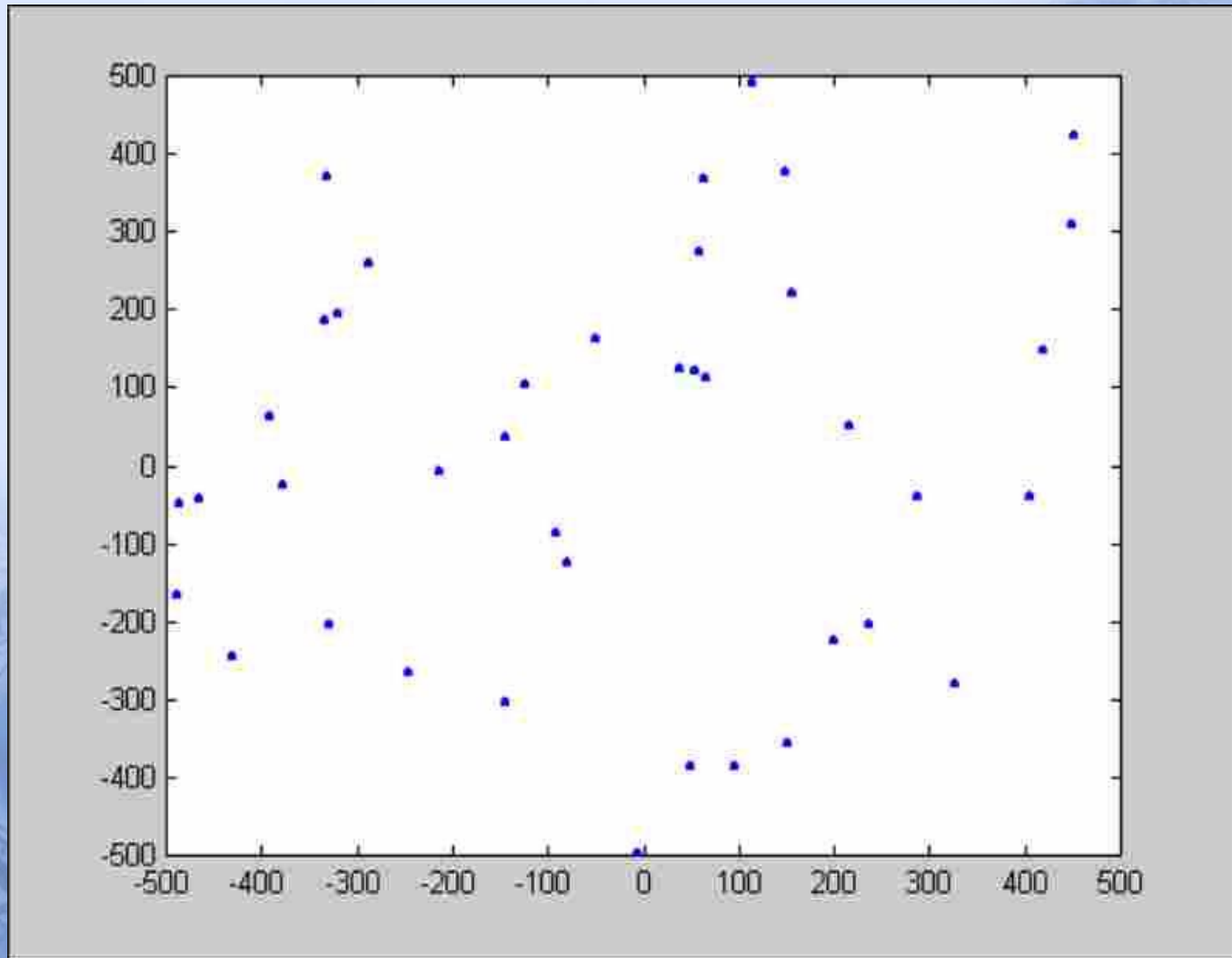


(6) 如果还没有到达结束条件，转到 (2)，否则输出最优解 *gBest* 并结束。

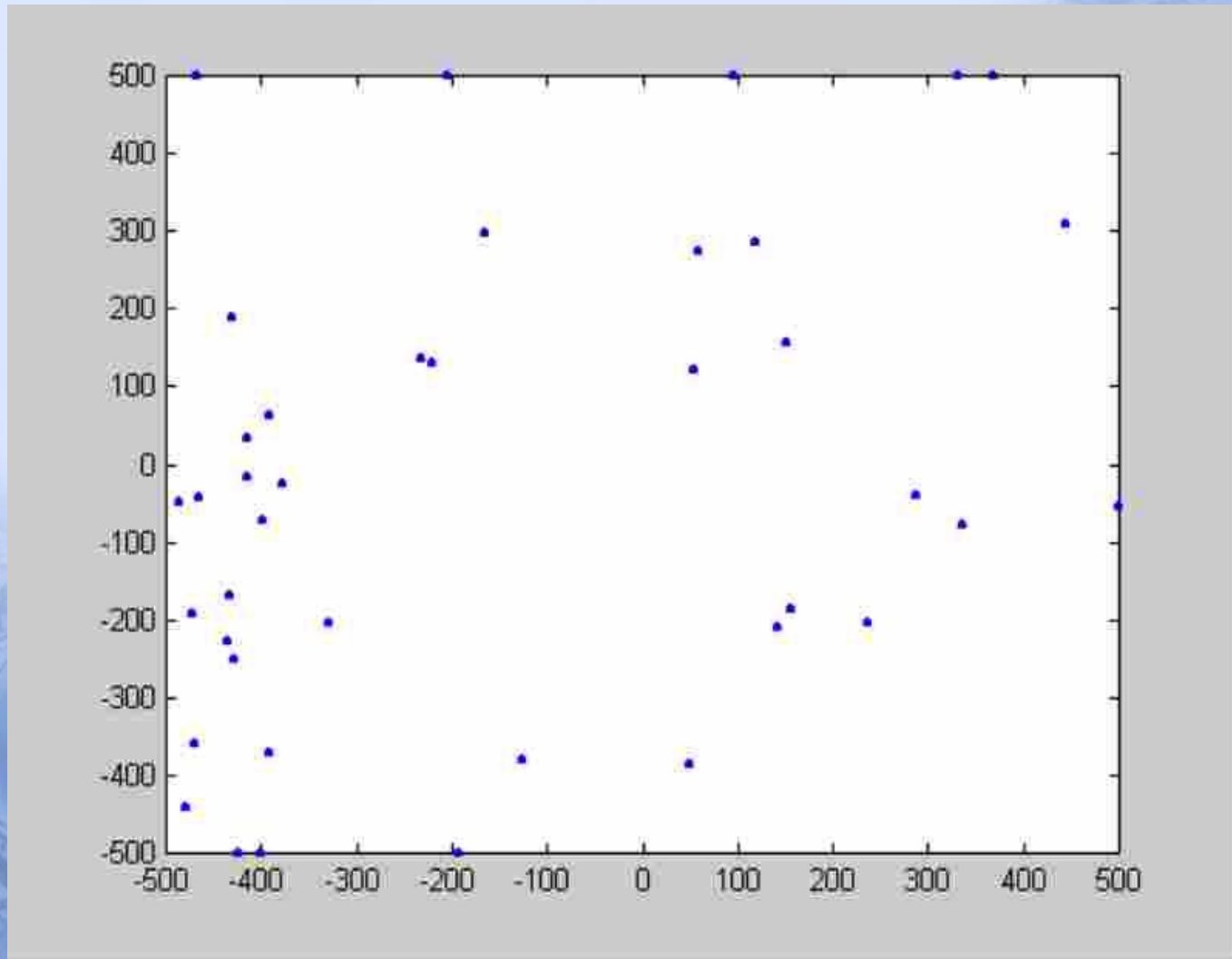
终止条件可以是：

- 达到最大迭代次数时终止。
- 找到一个可接受的解时终止。
- 连续多次迭代而解的状况没有改善时终止。
- 群体半径接近于零时终止。

# Initial State

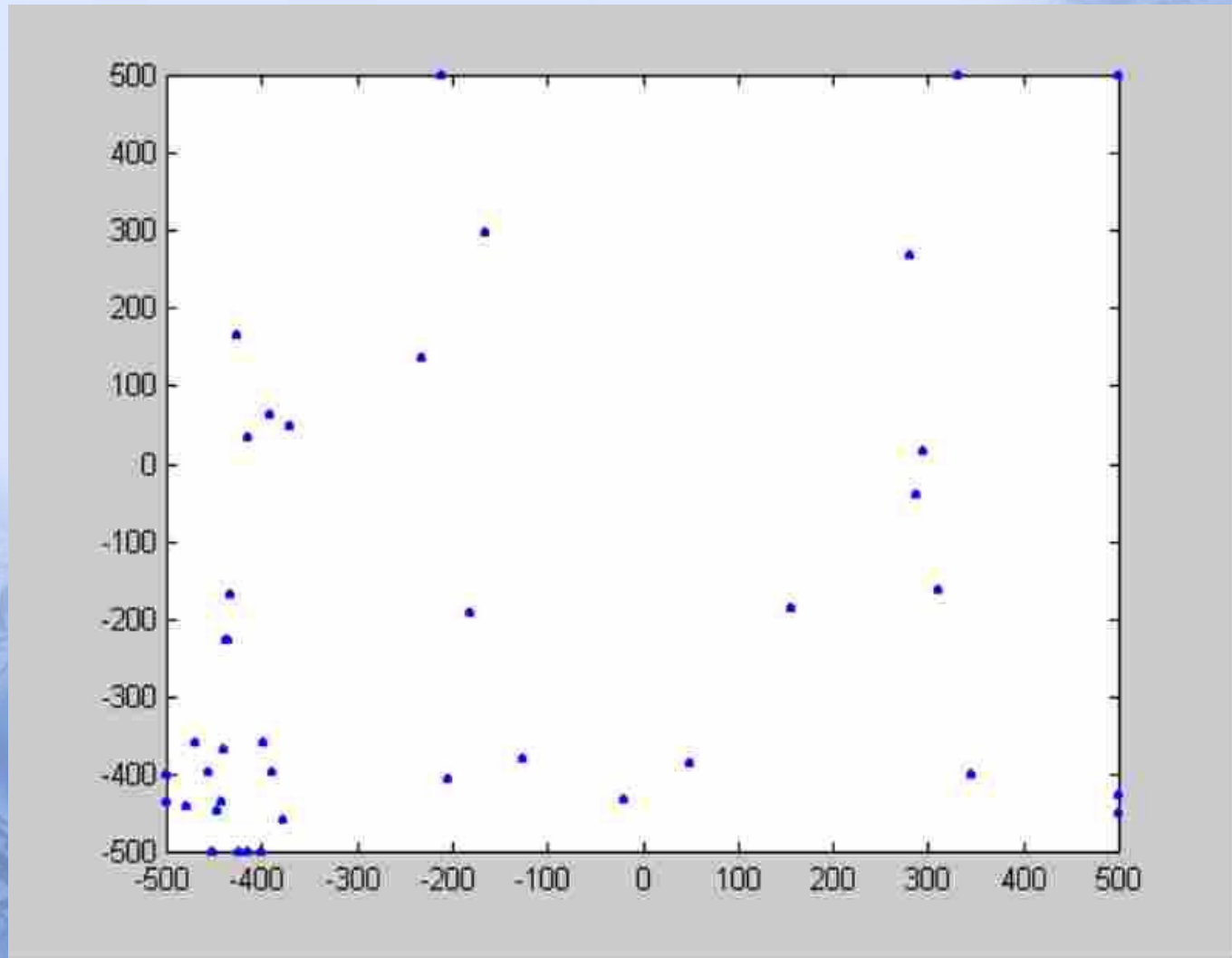


# After 5 Generations

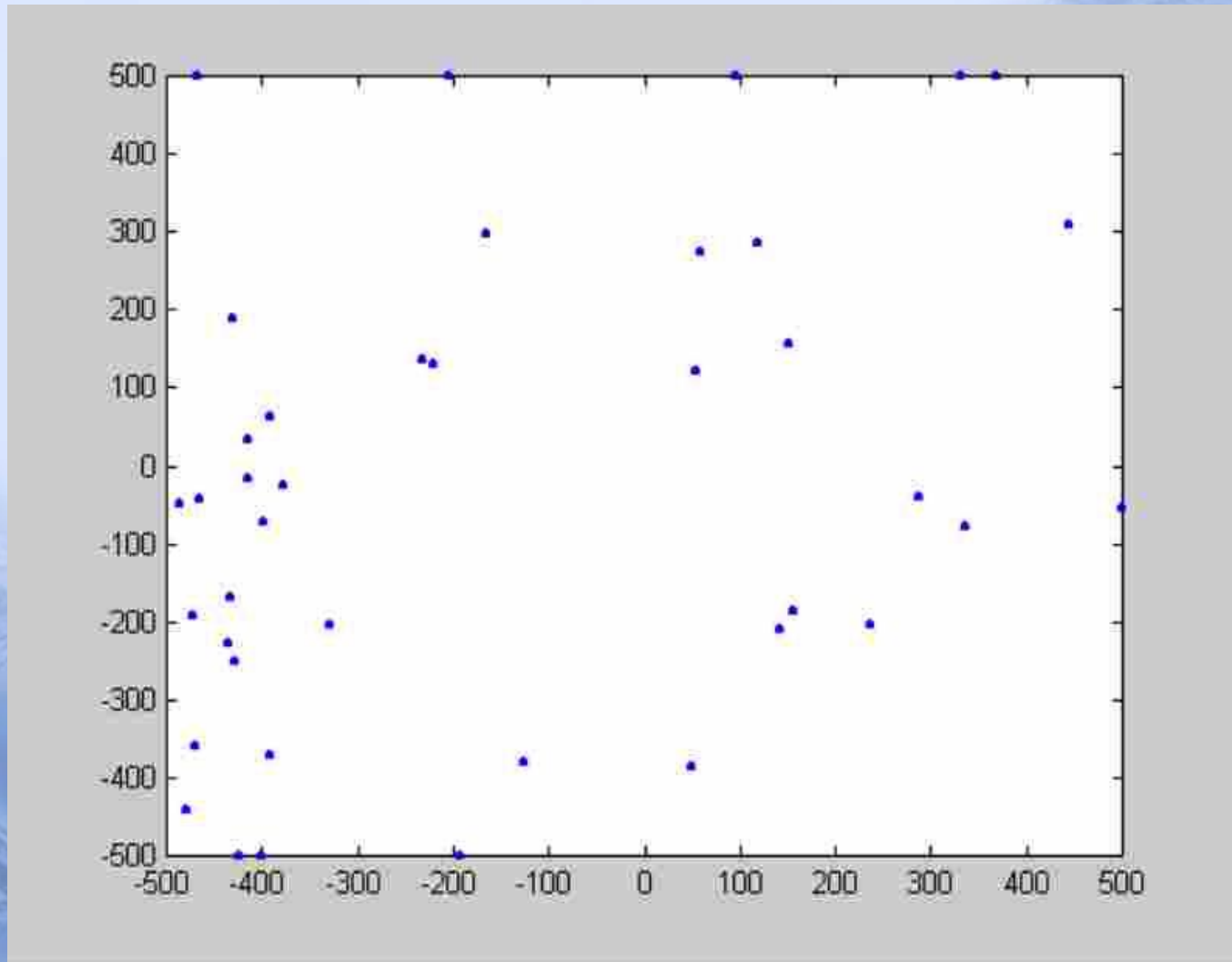




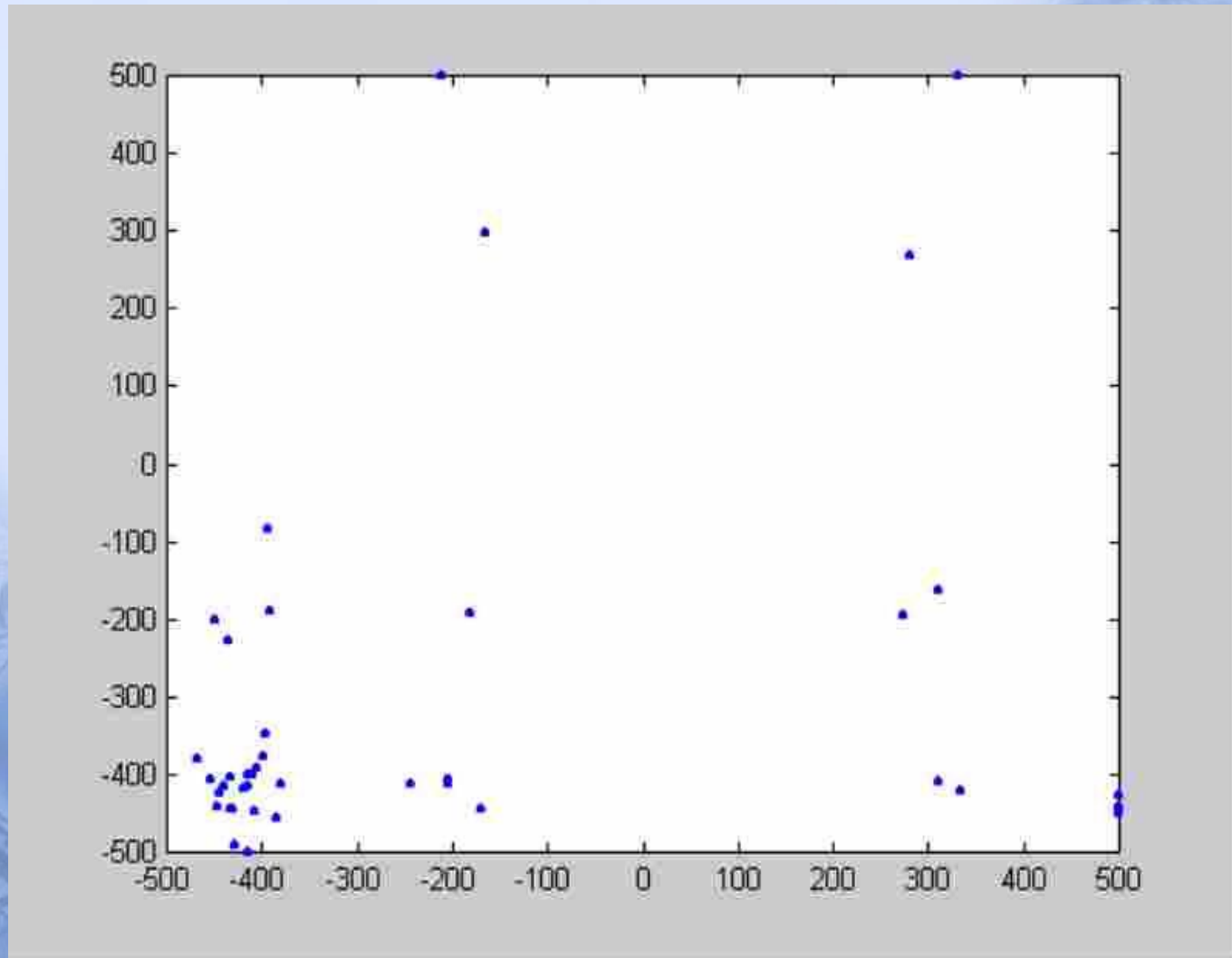
# After 10 Generations



# After 15 Generations



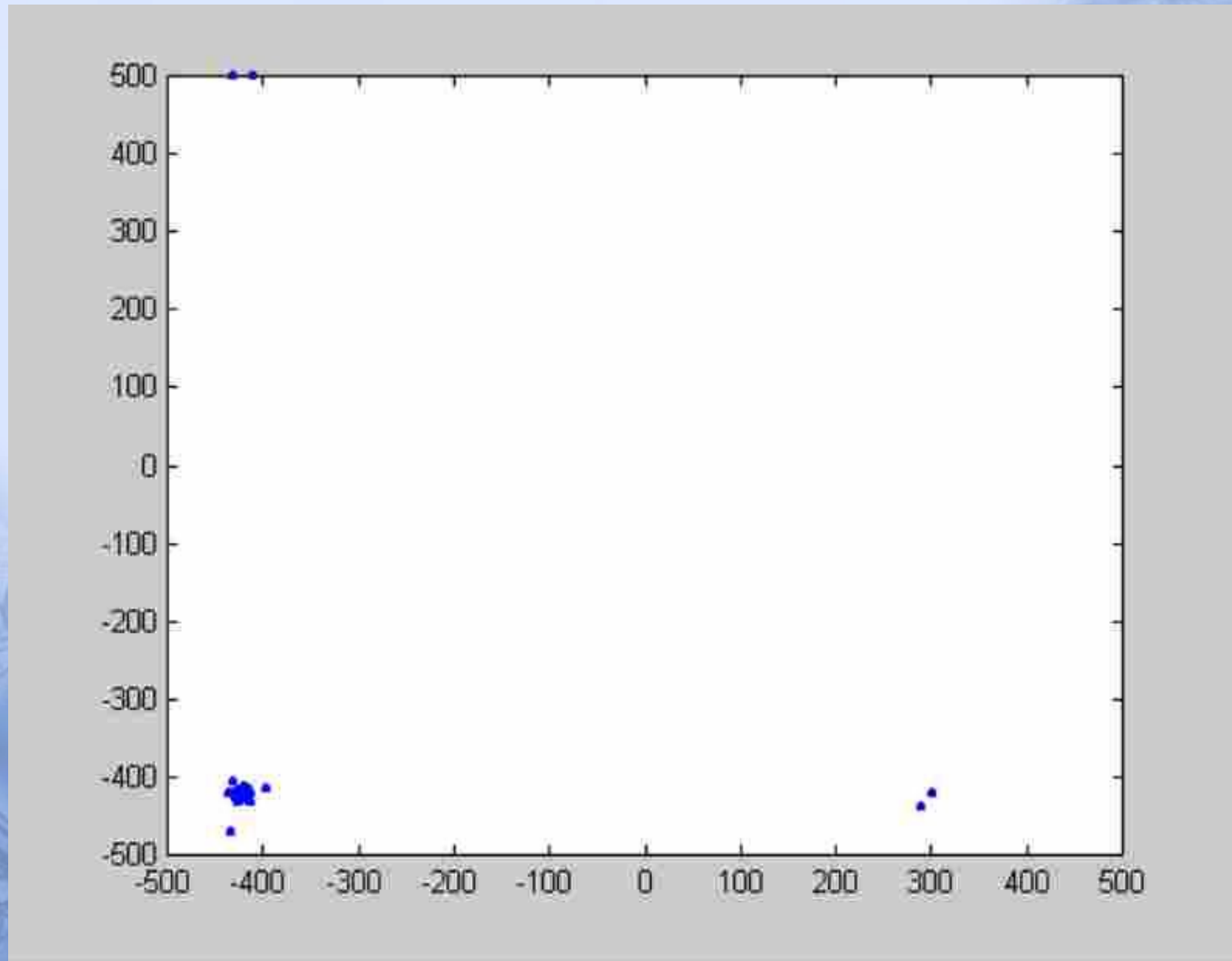
# After 20 Generations



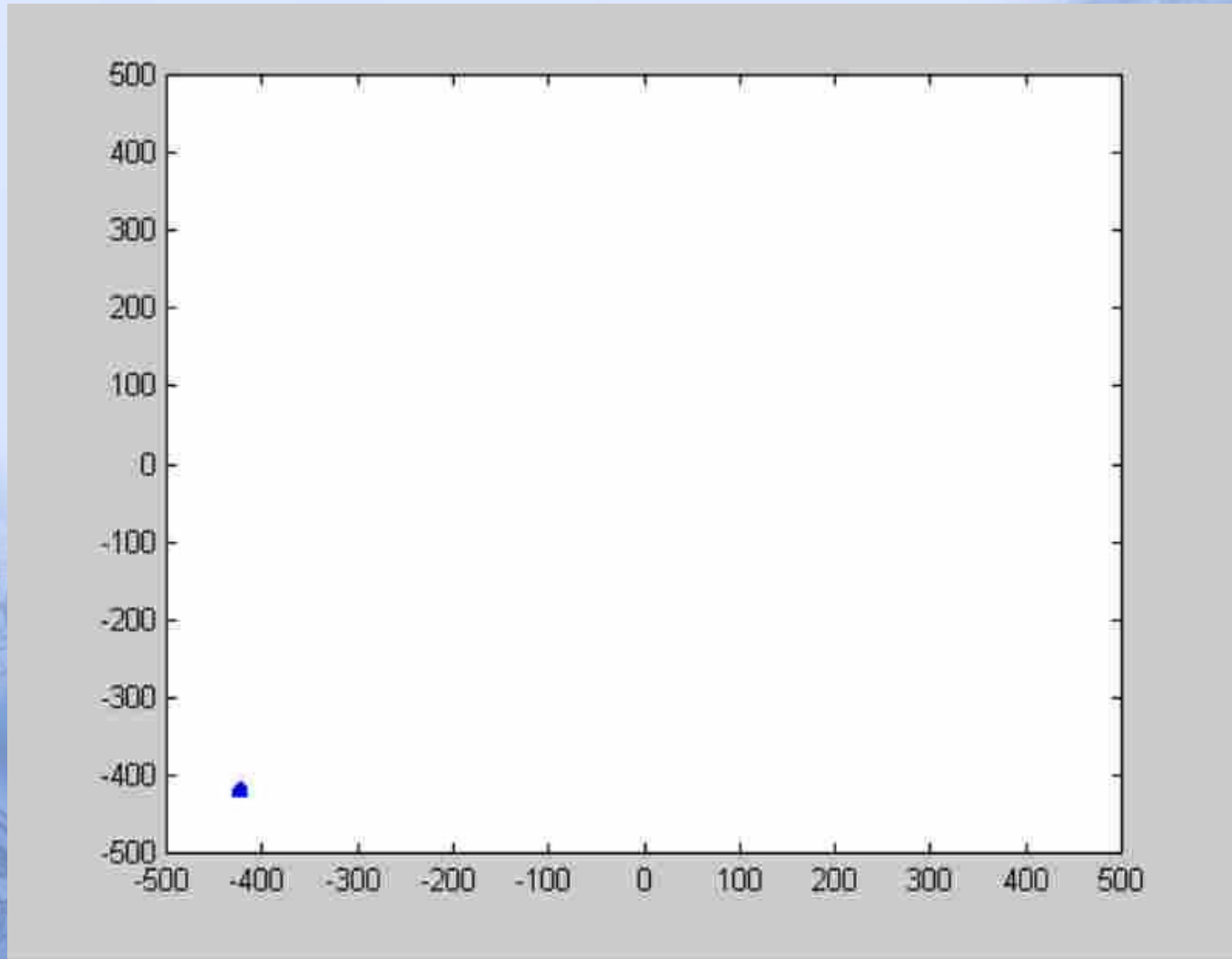


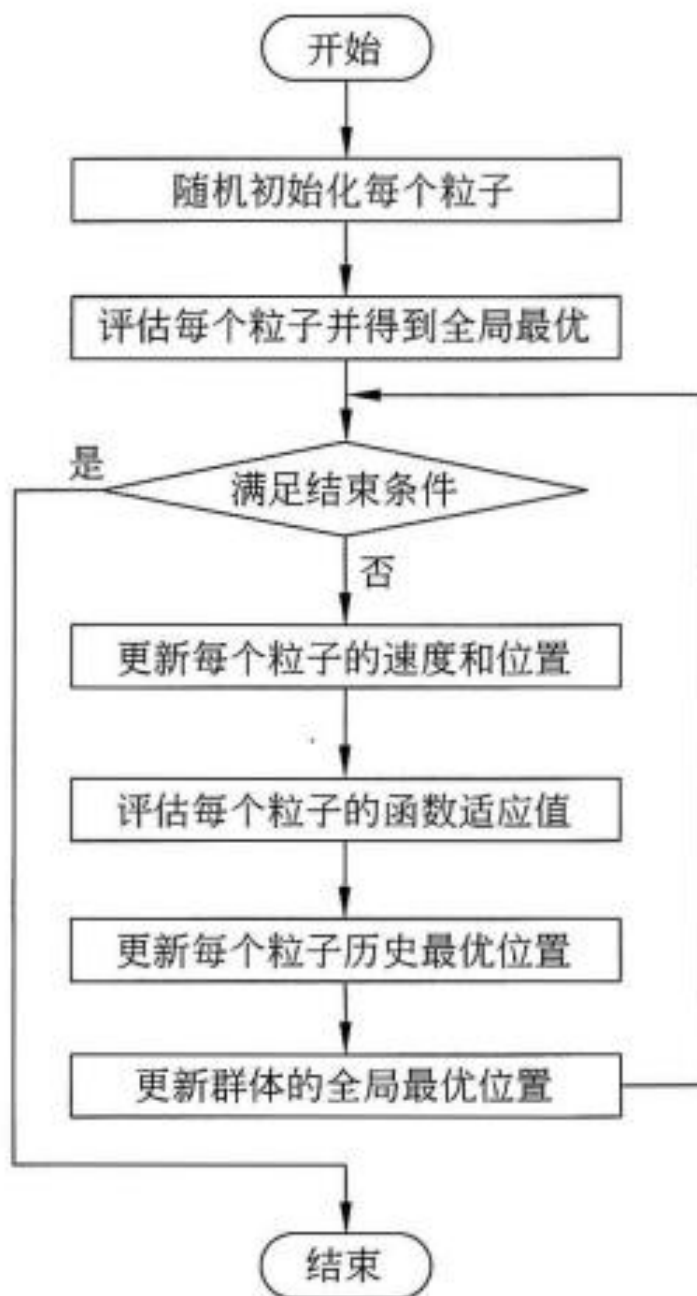


# After 100 Generations



# After 500 Generations





//功能：粒子群优化算法伪代码  
//说明：本例以求问题最小值为目标  
//参数：N为群体规模

**procedure** PSO

```
for each particle  $i$ 
    Initialize velocity  $V_i$  and position  $X_i$ 
    for particle  $i$ 
        Evaluate particle  $i$  and set  $pBest_i = X_i$ 
    end for
```

```
 $gBest = \min\{pBest_i\}$ 
```

```
while not stop
```

```
    for  $i=1$  to  $N$ 
```

```
        Update the velocity and position of
        particle  $i$ 
```

```
        Evaluate particle  $i$ 
```

```
        if  $\text{fit}(X_i) < \text{fit}(pBest_i)$ 
```

```
             $pBest_i = X_i;$ 
```

```
            if  $\text{fit}(pBest_i) < \text{fit}(gBest)$ 
```

```
                 $gBest = pBest_i;$ 
```

```
        end for
```

```
    end while
```

```
    print  $gBest$ 
```

```
end procedure
```



**例 6.1** 已知函数  $y=f(x_1, x_2)=x_1^2+x_2^2$ ,  
其中,  $-10 \leq x_1, x_2 \leq 10$ , 用粒子群优化算法求解  $y$  的最小值, 请写出关键的执行步骤。

步骤 1：初始化。

假设种群大小是  $N=3$ ；在搜索空间中随机初始化每个解的速度和位置，计算适应函数值，并且得到粒子的历史最优位置和群体的全局最优位置。

$$p_1 = \begin{cases} v_1 = (3, 2) \\ x_1 = (8, -5) \end{cases} \quad \begin{cases} f_1 = 8^2 + (-5)^2 = 64 + 25 = 89 \\ pBest_1 = x_1 = (8, -5) \end{cases}$$

$$p_2 = \begin{cases} v_2 = (-3, -2) \\ x_2 = (-5, 9) \end{cases} \quad \begin{cases} f_2 = (-5)^2 + 9^2 = 25 + 81 = 106 \\ pBest_2 = x_2 = (-5, 9) \end{cases}$$

$$p_3 = \begin{cases} v_3 = (5, 3) \\ x_3 = (-7, -8) \end{cases} \quad \begin{cases} f_3 = (-7)^2 + (-8)^2 = 49 + 64 = 113 \\ pBest_3 = x_3 = (-7, -8) \end{cases}$$

$$gBest = pBest_1 = (8, -5)$$



步骤 2：粒子的速度和位置更新。

根据自身的历史最优位置和全局的最优位置，更新每个粒子的速度和位置。

$$p_1 = \begin{cases} v_1 = \omega \times v_1 + c_1 \times r_1 \times (pBest_1 - x_1) + c_2 \times r_2 \times (gBest - x_1) \\ \Rightarrow v_1 = \begin{cases} 0.5 \times 3 + 0 + 0 = 1.5 \\ 0.5 \times 2 + 0 + 0 = 1 \end{cases} = (1.5, 1) \\ x_1 = x_1 + v_1 = (8, -5) + (1.5, 1) = (9.5, -4) \end{cases} \quad \begin{cases} v_1 = (3, 2) \\ x_1 = (8, -5) \end{cases}$$

$$p_2 = \begin{cases} v_2 = \omega \times v_2 + c_1 \times r_1 \times (pBest_2 - x_2) + c_2 \times r_2 \times (gBest - x_2) \\ \Rightarrow v_2 = \begin{cases} 0.5 \times (-3) + 0 + 2 \times 0.3 \times (8 - (-5)) = 6.1 \\ 0.5 \times (-2) + 0 + 2 \times 0.1 \times ((-5) - 9) = 1.8 \end{cases} = (6.1, 1.8) \\ x_1 = x_1 + v_1 = (-5, 9) + (6.1, 1.8) = (1.1, 10.8) = (1.1, 10) \end{cases} \quad \begin{cases} v_2 = (-3, -2) \\ x_2 = (-5, 9) \end{cases}$$



注意!

对于越界的位置, 需要进行合法性调整

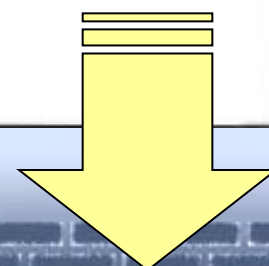
$$p_3 = \begin{cases} v_3 = \omega \times v_3 + c_1 \times r_1 \times (pBest_3 - x_3) + c_2 \times r_2 \times (gBest - x_3) \\ \Rightarrow v_3 = \begin{cases} 0.5 \times 5 + 0 + 2 \times 0.05 \times (8 - (-7)) = 3.5 \\ 0.5 \times 3 + 0 + 2 \times 0.8 \times ((-5) - (-8)) = 6.3 \end{cases} = (3.5, 6.3) \\ x_1 = x_1 + v_1 = (-7, -8) + (3.5, 6.3) = (-3.5, -1.7) \end{cases}$$

$$\begin{cases} v_3 = (5, 3) \\ x_3 = (-7, -8) \end{cases}$$

$\omega$  是惯量权重, 一般取  $[0, 1]$  区间的数, 这里假设为 0.5

$c_1$  和  $c_2$  为加速系数, 通常取固定值 2.0

$r_1$  和  $r_2$  是  $[0, 1]$  区间的随机数





步骤 3：评估粒子的适应度函数值。

更新粒子的历史最优位置和全局的最优位置。

$$f_1^* = 9.5^2 + (-4)^2 = 90.25 + 16 = 106.25 > f_1 = 89$$

$$\begin{cases} f_1 = 89 \\ pBest_1 = (8, -5) \end{cases}$$

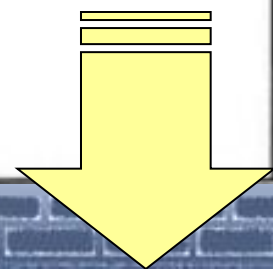
$$f_2^* = 1.1^2 + 10^2 = 1.21 + 100 = 101.21 < 106 = f_2$$

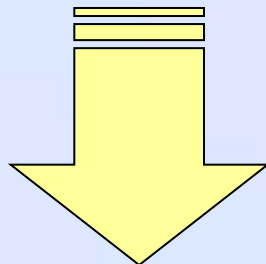
$$\begin{cases} f_2 = f_2^* = 101.21 \\ pBest_2 = X_2 = (1.1, 10) \end{cases}$$

$$f_3^* = (-3.5)^2 + (-1.7)^2 = 12.25 + 2.89 = 15.14 < 113 = f_3$$

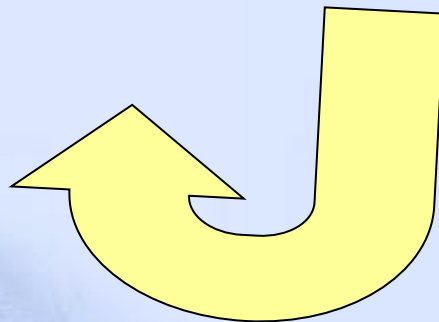
$$\begin{cases} f_3 = f_3^* = 15.14 \\ pBest_3 = x_3 = (-3.5, -1.7) \end{cases}$$

$$gBest = pBest_3 = (-3.5, -1.7)$$





步骤 4：如果满足结束条件，则输出全局最优结果并结束程序，  
否则，转向步骤 2 继续执行。



<https://aim.sustech.edu.cn/Article-index-id-58-typeid-6.html>

<https://www.sustech.edu.cn/zh/faculties/shiyuhui.html>

# 粒子群算法的改进研究

- 改进研究主要围绕五个方面展开：算法的理论研究、算法的参数研究、算法的拓扑结构研究、**PSO** 与其他算法的混合以及算法的应用研究

## 算法理论研究

指的是对粒子群优化算法进行理论分析,从数学推导上证明算法具有收敛性,能够保证算法求解到全局最优解。这种理论研究从早期的静态分析,已经逐步发展到动态分析,对保证算法的性能提供了理论依据。

希望从理论上证明算法的搜索能力和收敛能力。



### 算法参数研究

PSO 具有惯量权重  $\omega$ , 加速系数  $c_1$ 、 $c_2$ , 最大速度, 种群规模等参数。参数分析试图通过研究这些参数对算法全局搜索能力和局部搜索能力的影响, 找到更好的参数配置或者自适应调参方案, 提高算法性能。

希望从实验分析的角度理解算法运行机理, 提高算法性能。

### 拓扑结构研究

PSO 有全局版本和局部版本之分, 它们的不同之处在于更新速度的时候采用整个种群最优的粒子作为样本还是局部邻居中最优的粒子作为样本。不同的拓扑结构会影响算法局部搜索和全局搜索的能力。

希望通过研究不同的拓扑结构, 平衡算法的全局搜索和局部搜索能力。



## 混合算法研究

PSO 自身收敛速度快，但是容易落入局部最优。混合算法引入了进化算法中的相关算子，例如选择、交叉、变异等算子，或者其他的一些增加算法多样性的技术，提高算法性能。

希望通过加入一些附加的算子，提高种群的多样性以提高算法的性能。

## 算法应用研究

PSO 简单易用、性能高效，已经得到了广泛的应用。传统的应用以连续领域的优化问题居多，近年来，将 PSO 离散化，应用到离散组合优化问题中的研究也频频出现，算法应用范围进一步扩展。

希望通过将算法应用到各种不同的领域，在实践中检验算法的有效性。

# 理论研究改进

主要是关于 PSO 的数学基础、收敛性和稳定性研究：

|                     |                                      |
|---------------------|--------------------------------------|
| Clerc 与 Kennedy     | 设计了一个称为压缩因子的参数。在使用了此参数之后,PSO 能够更快地收敛 |
| Trelea              | 指出 PSO 最终稳定地收敛于空间中的某一个点,但不能保证是全局最优点  |
| Kadirkamanathan 等人  | 在动态环境中对 PSO 的行为进行研究,由静态分析深入到了动态分析    |
| F. van den Bergh 等人 | 对 PSO 的飞行轨迹进行了跟踪,深入到了动态的系统分析和收敛性研究   |

理论研究与分析,对巩固算法的理论基础有重要的意义,在理论研究的基础上,可以对粒子群优化算法的运行机理进一步深入了解和认识,对加强和改善算法的性能有实际的指导意义。

# 拓扑结构改进

- 粒子群算法的拓扑结构也称为社会结构，指的是**算法中的个体如何进行相互作用的问题**。
- 群体中的每个个体都在相互学习，除基于自身的认知之外，还在不断地**向比自己更好的邻居移动**。通过这样的信息交流，整个群体将能够聚集到一个全局最优的位置。
- PSO 的拓扑结构是由**相互重叠的邻域构成的**，而粒子就在这些邻域之内相互影响。
- 不同的拓扑结构的定义将影响着**粒子间信息交流的方式和信息流通的速度**，从而影响着算法的性能。



# 静态拓扑结构

在 Kennedy 和 Eberhart 提出 PSO 的时候，就提到了两个主要范式(paradigm):

全局版本PSO (Global Version PSO, **GPSO**)

局部版本 PSO (Local Version PSO , **LPSO**)

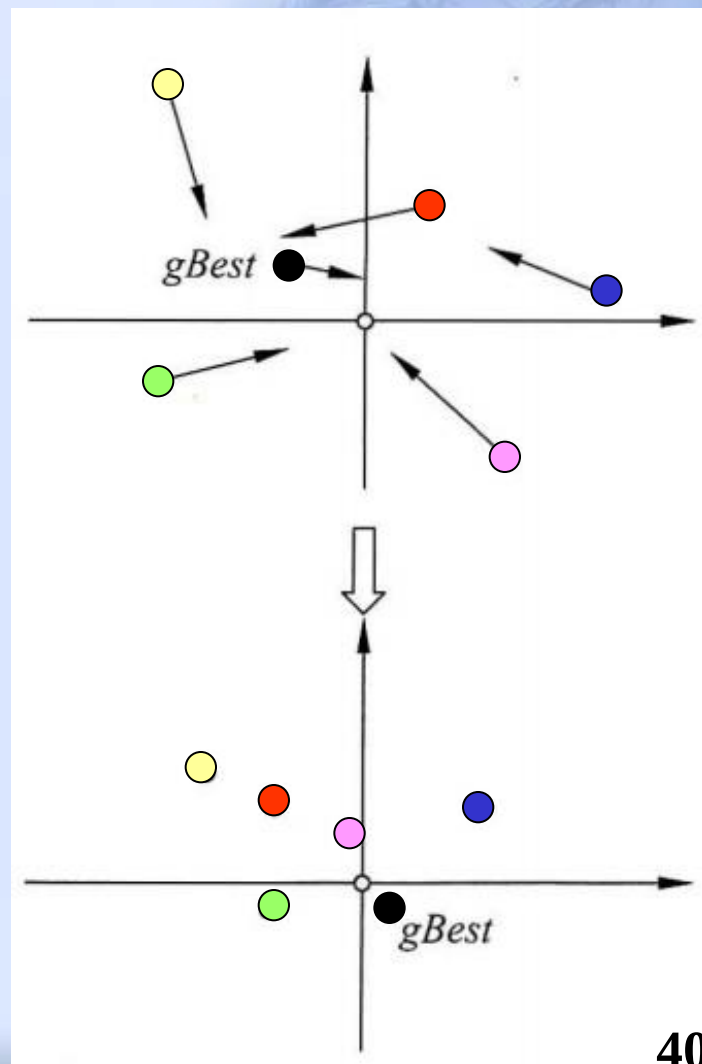
这两种范式的主要区别在于社会网络结构的不同。

- 在GPSO 中，**整个群体构成一个“社会”**，粒子在进行速度和位置更新的时候，将会使用自身的历史最好位置 **pBest** 和整个群体中最好的位置 **gBest** 作为更新的向导。
- 在 LPSO 中，每个粒子所处的 **“社会”** 仅仅是一个小的邻域。粒子在进行速度和位置更新时，除了使用自身的历史最好位置 **pBest** 之外，还要使用邻域中的最好位置 **lBest** 作为更新的向导



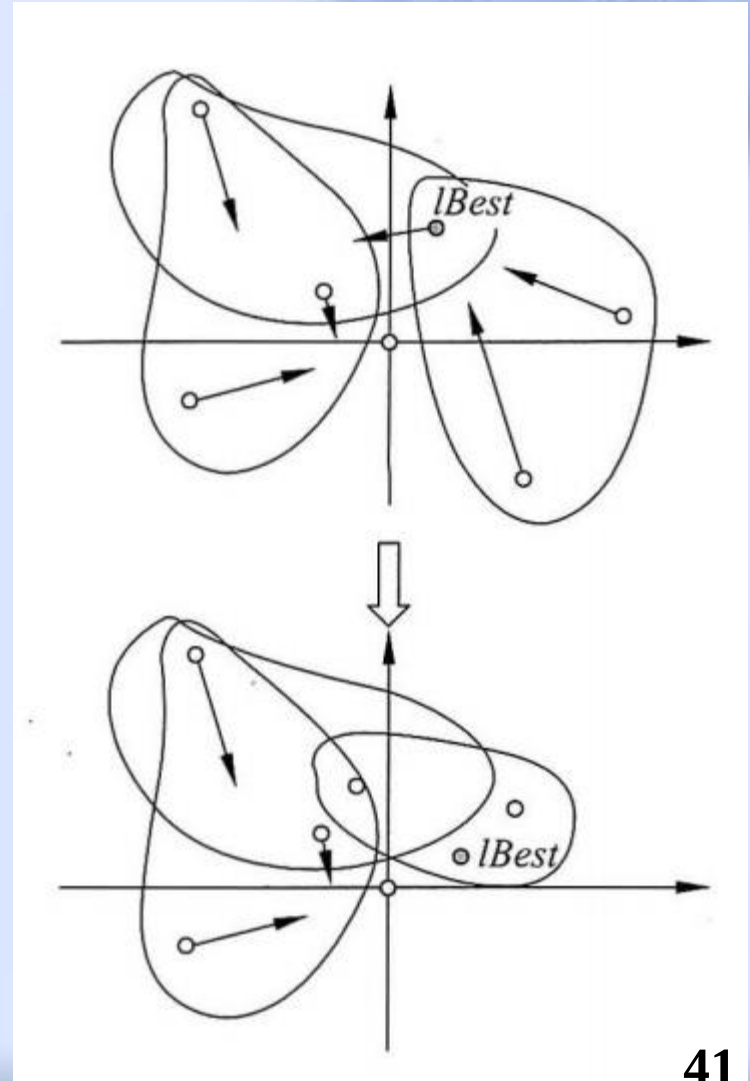
# GPSO

- 每个粒子受到群体中最优粒子 **gBest** 的影响。
- 每个粒子对应的 **gBest** 都是一样的。用来作为更新向导的位置比较单一。
- 所有粒子都将快速趋向该全局最优解。
- 这种快速的收敛容易导致算法落入局部最优



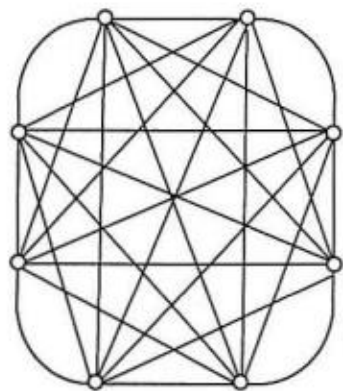
# LPSO

- 每个粒子受到其邻域结构中最优的粒子 **lBest** 的影响，
- 每个粒子对应的 **lBest** 很可能是不同的，用作更新向导的位置将要比 **GPSO** 要多
- 整个群体表现出更好的多样性，往往能够在处理复杂的问题时表现出比 **GPSO** 更好的性能。
- 不容易受到当前全局最优解的过分影响而落入局部最优

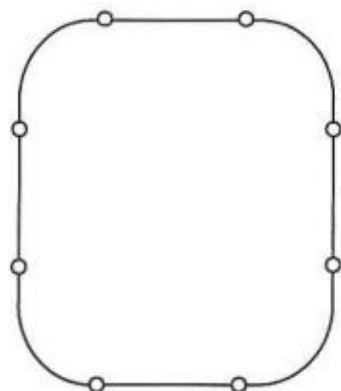


## 不同拓扑结构特点比较

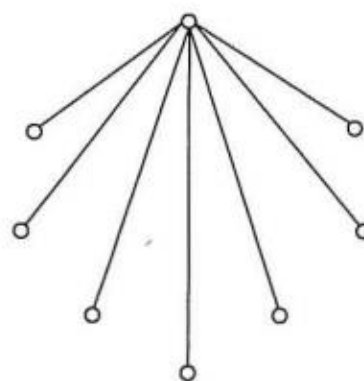
| 拓 扑 名 称 | 结 构 特 点                 |
|---------|-------------------------|
| 星型结构    | 任意两个粒子都相互连接             |
| 环型结构    | 每个粒子和左右两个粒子相连           |
| 齿形结构    | 邻域中仅有一个粒子作为“焦点”，和其他粒子相连 |
| 冯·诺依曼结构 | 每个粒子和平面网格中的上下左右四个粒子相连   |



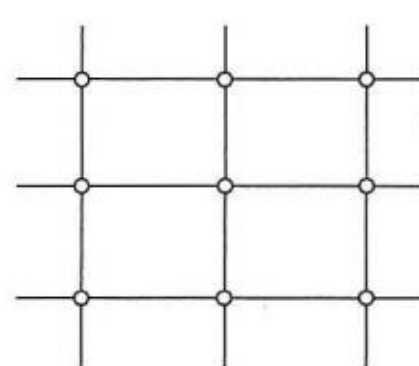
(a) 星型结构



(b) 环型结构



(c) 齿形结构



(d) 冯·诺依曼结构

- 全局版本PSO和局部版本 PSO的收敛特点

- (1) GPSO 由于具有**很高的连接度**，往往具有比 LPSO **更快的收敛速度**。但是，快速的收敛也让 GPSO 付出了多样性迅速降低的代价。
- (2) LPSO 由于具有**更好的多样性**，因此一般不容易落入局部最优，在**处理多峰问题上具有更好的性能**。

- 在解决具体问题的时候，可以遵循以下一些规律：

- (1)邻域较小的拓扑结构在处理**复杂的、多峰值的问题**上具有优势，例如环型结构的 LPSO
- (2) 随着邻域的扩大，算法的收敛速度将会加快，这对**简单的、单峰值的问题**非常的有利，GPSO 在这些问题上表现很好
- (3) 冯·诺依曼结构在GPSO 和 LPSO 两者间**取得了一个平衡**，适合大多数问题的求解，而且具有良好的性能



# 动态拓扑结构

动态拓扑结构是希望通过各种方法提高算法的整体性能:

- 在不同的进化阶段使用不同的拓扑结构,
- 动态地改变算法的探索能力和开发能力,
- 在保持种群多样性和算法收敛性上取得动态的变化和平衡

粒子群优化算法动态拓扑结构研究一览表

| 特 点                                  |
|--------------------------------------|
| 逐步增长法: 根据进化代数将其邻居从粒子自身逐渐扩大到整个群体      |
| 最小距离法: 每一代中选择 $m$ 个“距离”最近的粒子作为邻居     |
| 重新组合法: 随机分成多个小种群进化了一定代数后重新随机组合, 继续进化 |
| 随机选择法: 随机为每个粒子选择 $K$ 个粒子(包括自身)作为邻域   |

## 标准的PSO算法 (Standard PSO)

- 随机拓扑结构 PSO( Random Topology PSO, RPSO)为每个粒子定义一个规模为  $K$  的邻域, 该邻域内的个体是从整个群体中随机选择的(包括粒子本身)。
- 在每一次迭代中, 每个粒子将在其本身历史最优位置和其对应的随机拓扑邻域中的最优位置的影响下进行速度和位置的更新。
- 如果在该次迭代中得到的最优解对已经找到的全局最优解有所改善, 那么这些拓扑结构将不会发生改变而在下一代继续发挥作用。
- 否则, RPSO 将在下一代中对每个粒子重新构造其随机邻域, 以期增强粒子的搜索能力, 提高算法的性能。

# 离子群算法的参数设置

|           |                                                                                                                                                                                                                                                       |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 种群规模 $N$  | <ul style="list-style-type: none"><li>✓ 影响着算法的搜索能力和计算量</li><li>• PSO 对种群规模要求不高,一般取 20~40 就可以达到很好的求解效果<sup>[96]</sup></li><li>• Carlisle 和 Dozier<sup>[97]</sup>的研究中建议将 <math>N</math> 设为 30</li><li>• 不过对于比较难的问题或者特定类别的问题,粒子数可以取到 100 或 200</li></ul> |
| 粒子的长度 $D$ | <ul style="list-style-type: none"><li>✓ 由优化问题本身决定,就是问题解的长度</li></ul>                                                                                                                                                                                  |
| 粒子的范围 $R$ | <ul style="list-style-type: none"><li>✓ 由优化问题本身决定,每一维可以设定不同的范围</li></ul>                                                                                                                                                                              |



## 最大速度 $V_{\max}$

- ✓ 决定粒子每一次的最大移动距离,制约着算法的探索 and 开发能力
  - $V_{\max}$  的每一维  $V_{\max}^d$  一般可以取相应维搜索空间的  $10\% \sim 20\%$ , 甚至  $100\%$
  - 在文献[98]中,就提出了一种将  $V_{\max}$  按照进化代数从大到小递减的设置方案

## 惯性权重 $\omega$

- ✓ 控制着前一速度对当前速度的影响,用于平衡算法的探索 and 开发能力
  - 一般设置为从  $0.9$  线性递减到  $0.4$  [8][96], 也有非线性递减的设置方案
  - 可以采用模糊控制的方式设定 [99], 或者在  $[0.5, 1.0]$  之间随机取值 [100]
  - $\omega$  设为  $0.729$  的同时将  $c_1$  和  $c_2$  设为  $1.49445$ , 有利于算法的收敛 [96]

## 压缩因子 $\chi$

- ✓ 限制粒子的飞行速度的, 保证算法的有效收敛
  - Clerc 等人 [27] 通过数学计算得到  $\chi$  取值  $0.729$ , 同时  $c_1$  和  $c_2$  设为  $2.05$



## 加速系数 $c_1$ 和 $c_2$

- ✓ 代表了粒子向自身极值  $pBest$  和全局极值  $gBest$  推进的加速权值
  - $c_1$  和  $c_2$  通常都等于 2.0, 代表着对两个引导方向的同等重视<sup>[1]</sup>
  - 也存在一些  $c_1$  和  $c_2$  不相等的设置<sup>[33][97]</sup>, 但其范围都在 0~4 之间
  - 将  $c_1$  线性减少,  $c_2$  线性增大的设置能动态平衡算法的多样性和收敛性<sup>[13]</sup>
  - 研究对  $c_1$  和  $c_2$  的自适应调整方案对算法性能的增强有重要意义<sup>[101-103]</sup>

## 终止条件

- ✓ 决定算法运行的结束, 由具体的应用和问题本身确定
  - 将最大循环数设定为 500、1000、5000, 或者最大的函数评估次数等
  - 也可以使用算法求解得到一个可接受的解作为终止条件
  - 或者是当算法在很长一段迭代中没有得到任何改善, 则可以终止算法

## 全局和局部 PSO

- ✓ 决定算法如何选择两种版本的粒子群优化算法——全局版 PSO 和局部版 PSO
  - 全局版本 PSO 速度快,不过有时会陷入局部最优
  - 局部版本 PSO 收敛速度慢一点,不过不容易陷入局部最优
  - 在实际应用中,可以根据具体问题选择具体的算法版本

## 同步和异步更新

- ✓ 两种更新方式的区别在于对全局的 *gBest* 或者局部的 *lBest* 的更新方式
  - 在同步更新方式中,在每一代中,当所有粒子都采用当前的 *gBest* 进行速度和位置的更新之后才对粒子进行评估,更新各自的 *pBest*,再选最好的 *pBest* 作为新的 *gBest*
  - 在异步更新方式中,在每一代中,粒子采用当前的 *gBest* 进行速度和位置的更新,然后马上评估,更新自己的 *pBest*,而且如果其 *pBest* 要优于当前的 *gBest*,则立刻更新 *gBest*,迅速将更好的 *gBest* 用于后面的粒子的更新过程中
  - 一般而言,异步更新的 PSO 具有高效的信息传播能力,具有更快的收敛速度



# 混合进化算子

## 选择算子 (Selection)

Angeline(1998)<sup>[38]</sup>将每次迭代产生的新的粒子群根据适应函数进行选择,用适应度较高的一半粒子的位置和速度矢量取代适应度较低的一半粒子的相应矢量,而保持后者个体极值不变

## 交叉算子 (Crossover)

Lovbjerg 等人(2001)<sup>[39]</sup>,Chen 等人(2007)<sup>[40]</sup>

## 变异算子 (Mutation)

Andrews(2006)<sup>[41]</sup>对 PSO 中混合使用变异算子的算法进行了较为全面的综述和比较

# 混合其它搜索算法

## 结合差分进化算法 (Differential Evolution)

Hendtlass(2001)<sup>[42]</sup>对同一个种群轮流地使用 PSO 和 DE 进行优化,结果表明这种方式比单纯的 PSO 具有更好的性能

Zhang 和 Xie(2003)<sup>[43]</sup>针对粒子的历史最优 *pBest* 利用差分操作进行扰动,有点类似于变异,而且是当且仅当扰动后的位置比当前的 *pBest* 更优时,才替换 *pBest*

## 结合局部搜索算法 (Local Search)

Liang 和 Suganthan(2005)<sup>[44]</sup>在设计动态的多种群 PSO 算法的时候,增强了种群的多样性,但是牺牲了快速收敛的能力,因此引入了准牛顿法对一部分适应值较好的 *lBest* 执行局部搜索操作



# 混合其它技术

## 函数延伸技术 (Function Stretching)

Parsopoulos 等人(2004)<sup>[11]</sup>将自适应改变目标函数方法用到了粒子群优化算法中,以防止粒子群优化算法返回已经找到的局部极值

## 混沌技术 (Chaos Technique)

Chuanwen 和 Bompard (2005)<sup>[45]</sup>以及 Coelho 和 BM Herrera (2007)<sup>[46]</sup>,使用混沌技术生成 PSO 相应的参数,包括惯量权重和加速系数等

Liu 等人(2005)<sup>[47]</sup>采取混沌技术对当前的最优位置进行扰动,使得最优解质量进一步提高

## 量子技术 (Quantum Technique)

Sun 等人(2004)<sup>[48]</sup>以及 Mikki 和 Kishk(2006)<sup>[49]</sup>分别提出了一种具有“量子行为”的 QDPSO,他们的算法改变了传统 PSO 在牛顿空间中的运动形式,而采用了量子式的更新方式对粒子的位置进行调整和优化

Yang 等人(2004)<sup>[50]</sup>将量子的思想应用于二进制版本的 PSO 中

## 小生境技术 (Niche Technique)

Brits 等人(2002)<sup>[52]</sup>和(2007)<sup>[53]</sup>提出的 NichePSO 中,他们采用了一个大的种群开始在问题空间中寻优,当发现了一个可能的最优解或者局部最优解的时候,种群自适应的分裂,让一小部分个体在该解附近形成一个小生境继续优化,而剩下的部分继续在解空间中寻优,直到发现下一个最优解的时候再进行自适应分裂,通过这样的方式,NichePSO 可以有效地搜索到多个最优解



## 协同技术 (Cooperative Technique)

Van den Bergh 和 Engelbrecht(2004)<sup>[12]</sup>将  $D$  维向量分到  $D$  个独立粒子群中,每个粒子群优化一维向量,评价适应值时将分量合成一个完整向量

Baskar 和 Suganthan(2004)<sup>[51]</sup>使用  $S$  个独立的粒子群,其中前  $S-1$  个粒子群根据“本粒子群”迄今搜索到的最优解来更新群中粒子的速度,而第  $S$  个粒子群则是根据“全部粒子群”迄今搜索到的最优解更新群中粒子的速度